

Méthode des Éléments de Frontière (BEM) appliquée à l'équation de Helmholtz 2D

Clotilde Auber–Joignant, Antoine Levrat

1 Introduction

Ce travail s'inscrit dans la continuité des précédents compte-rendus, dans lesquels nous avons introduit le problème de diffraction d'une onde par un obstacle et une façon de résoudre ce type de problèmes via la méthode des éléments de frontière.

La résolution de l'équation de Helmholtz par la méthode des éléments de frontière (BEM) conduit à la résolution d'un système linéaire du type :

$$Ax = b,$$

où A est une matrice dense. Cette densité entraîne des coûts mémoire et temps de calcul en $O(N^2)$ pour un solveur itératif, et en $O(N^3)$ pour une factorisation directe.

Afin de rendre la résolution de tels systèmes accessible pour des problèmes de tailles réalistes, il est nécessaire d'introduire une approximation de la matrice A qui réduise la complexité des calculs. Pour cela, nous utiliserons la méthode des matrices hiérarchiques, qui permet de représenter de grandes matrices de manière compacte et d'effectuer des calculs rapides tout en contrôlant la précision de l'approximation.

L'objectif est ensuite de comparer les temps de calcul de notre code BEM appliqué à un problème de diffraction lorsque la matrice A est calculée explicitement, et lorsque la matrice A est hiérarchique. Nous explorerons également comment passer à d'autres géométries, dans des cas de diffraction avec des obstacles différents d'un disque.

2 Méthode des matrices hiérarchiques (H-mat)

→ *Pourquoi la méthode des matrices hiérarchiques est-elle pertinente ici ?*

Nous disposons d'une matrice A , qui est dense, mais dont nous connaissons des propriétés qui nous permettent d'en déduire la structure. En effet, les lignes et colonnes de A correspondent aux degrés de liberté de notre système. Plus les points ont des indices proches dans la matrice et plus ces points sont proches physiquement dans notre géométrie. De plus, la matrice A est construite à partir d'une fonction de Green, donc nous nous attendons à ce que des singularités apparaissent lorsque la distance entre deux (ensembles de) points est faible. En particulier, nous nous attendons à observer ce comportement sur la diagonale de la matrice.

Nous nous retrouvons ainsi dans une situation où la précision de l'approximation de A dépend de la distance entre deux (ensembles de) points. La méthode des matrices

hiérarchiques nous permet de construire un algorithme qui confronte la distance entre deux ensembles de points, que nous appellerons désormais 'boîtes', à un critère d'admissibilité. En fonction de ce critère, nous effectuerons soit une approximation de rang faible du bloc correspondant (si la distance est jugée suffisante), soit le bloc sera calculé explicitement (si la distance est trop faible). La sensibilité du critère d'admissibilité permet d'ajuster la précision de l'approximation, ce que nous détaillerons en section 2.2.

2.1 Découpage de la géométrie et construction d'une hiérarchie spatiale

La première étape pour appliquer la méthode H-mat est de construire une hiérarchie spatiale, c'est-à-dire de définir une façon de rassembler les points. Pour cela, on effectue une dichotomie de la géométrie afin de construire des boîtes contenant des points physiquement proches. Nous avons effectué cette dichotomie de deux façons : selon un critère géométrique (couper le milieu de la boîte englobante selon l'axe le plus long) et selon un critère d'équilibrage (afin que les deux sous boîtes contiennent un nombre similaire de points), dans le cas du disque mais aussi pour des géométries générées aléatoirement, comme représenté en Figure 2.

Dans la suite, nous nous sommes basés sur un découpage géométrique du disque, comme représenté ci-dessous (Figure 1). Nous avons également représenté l'arbre binaire associé afin de pouvoir par la suite vérifier visuellement quelles boîtes étaient apparentées afin de tester notre critère d'admissibilité.

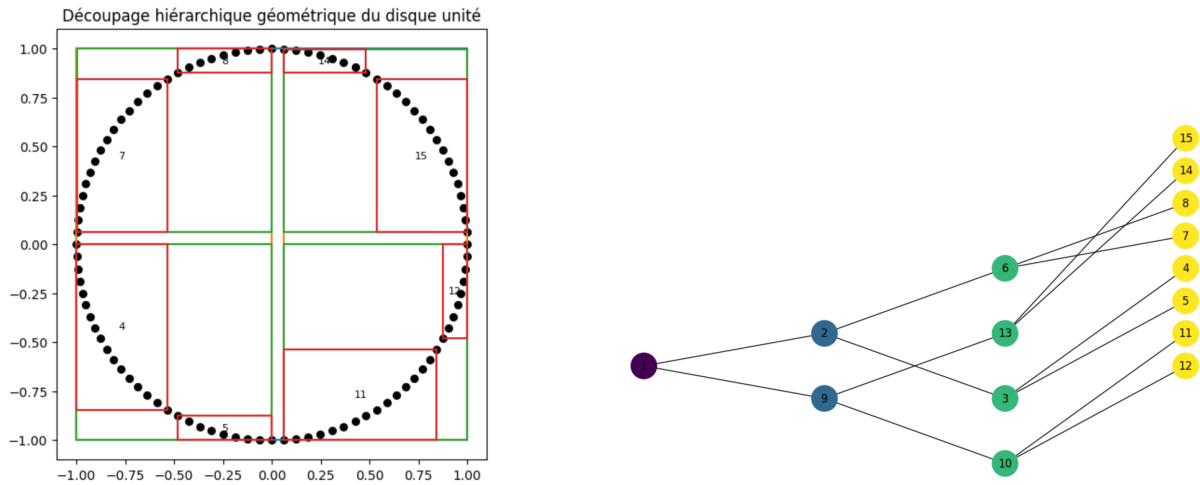


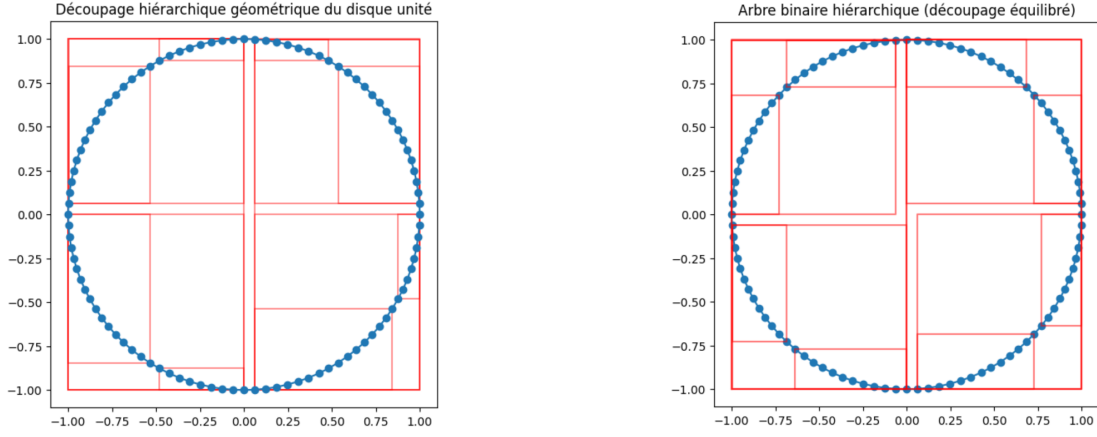
Figure 1 – Découpage géométrique du disque (100 points, niveau max = 3) et arbre binaire associé.

Les feuilles sont ordonnées verticalement de manière à correspondre à la proximité physique des boîtes*, et non à leur numérotation qui est arbitraire. Cela permet d'obtenir une représentation plus lisible de la structure de la matrice hiérarchique en rassemblant les blocs non admissibles sur la diagonale (cf section 2.2).

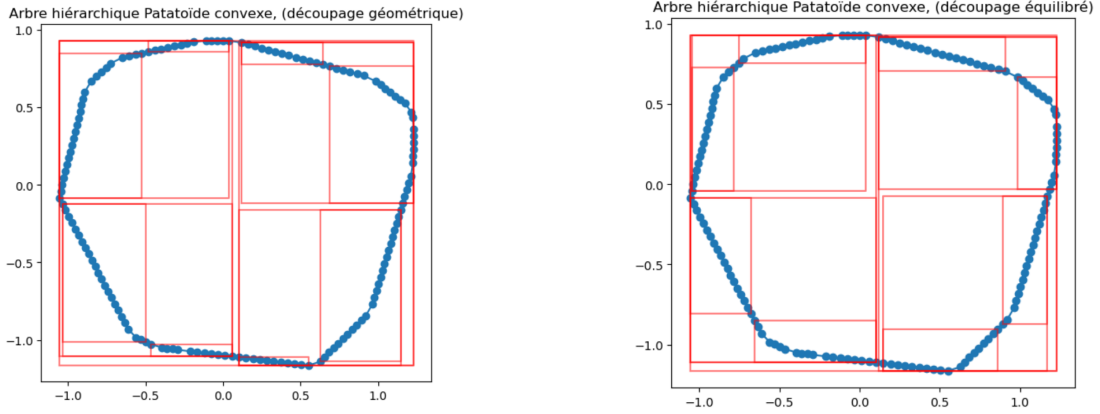
**Remarque : L'ordre correspond à celui obtenu en parcourant le disque dans le sens anti-horaire en partant de (1,0). En pratique, nous avons implémenté cela en calculant l'angle theta associé au centre de chaque boîte et en les triant par ordre croissant.*

Figure 2 – Découpage hiérarchique géométrique (à gauche) et équilibré (à droite) comportant 15 boîtes, appliqué au disque (100 points) et à deux géométries générées aléatoirement (patate convexe avec 80 points et patate non convexe avec 60 points)

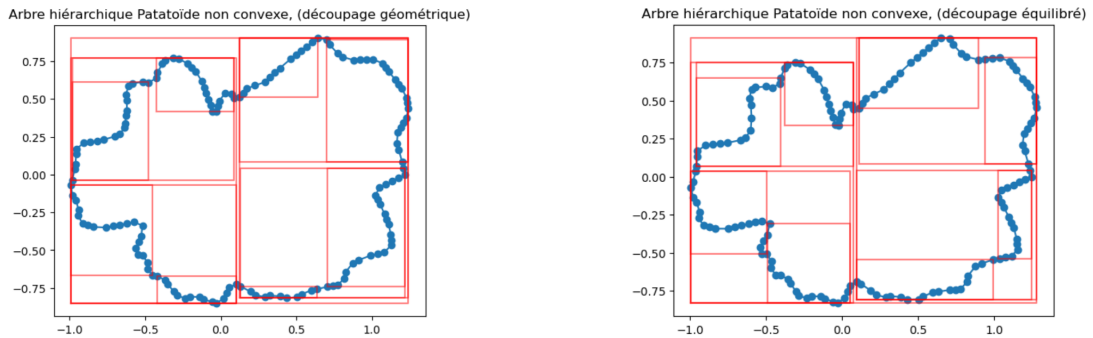
Disque



Patate convexe



Patate non convexe



Chaque méthode de découpage possède ses avantages et ses inconvénients. Le découpage géométrique est intuitif, facile à implémenter et reflète réellement la proximité physique des points. En contrepartie, l'arbre peut être déséquilibré si les points ne sont pas uniformément répartis. Le découpage équilibré a plus de sens dans un contexte de parallélisation, car il répartit mieux la charge de calcul. En contrepartie, des points physiquement proches peuvent se trouver dans des boîtes admissibles, ce qui nuit à la qualité d'approximation.

Une autre approche que l'on pourrait considérer serait une méthode hybride, qui prendrait en compte à la fois l'aspect géométrique et le nombre de points. Cette méthode serait

un bon compromis entre parallélisation et précision, mais serait plus compliquée à implémenter.

2.2 Critère d'admissibilité *a priori*

L'utilisation d'un critère d'admissibilité *a priori* permet d'identifier quels blocs sont de rang faible et peuvent donc être approximés par ACA partiel, sans avoir à calculer leur rang (ce qui reviendrait en termes de coût de calcul à calculer explicitement la matrice A, et que l'on cherche à éviter). Le critère utilisé pour déterminer si deux boîtes X et Y sont admissibles est :

$$\min(\text{diam}(X), \text{diam}(Y)) < \eta \text{dist}(X, Y), \quad \eta = 3.$$

Autrement dit, si X et Y sont suffisamment petites et éloignées l'une de l'autre, alors le bloc de matrice correspondant peut être compressé.

Plus η est grand et plus le nombre de blocs admissibles sera important. La compression sera plus forte, et donc la précision de l'approximation sera moins bonne. À l'inverse, plus η est petit et plus le critère est strict, seuls les blocs très petits et éloignés seront admissibles. La précision est meilleure mais le coût de stockage et les temps de calculs seront plus importants.

Nous avons représenté graphiquement la structure de la matrice hiérarchique obtenue à partir de l'arbre binaire construit en section 2.1, comme présenté en Figure 3. Les blocs admissibles sont affichés en vert et les blocs denses en rouge.

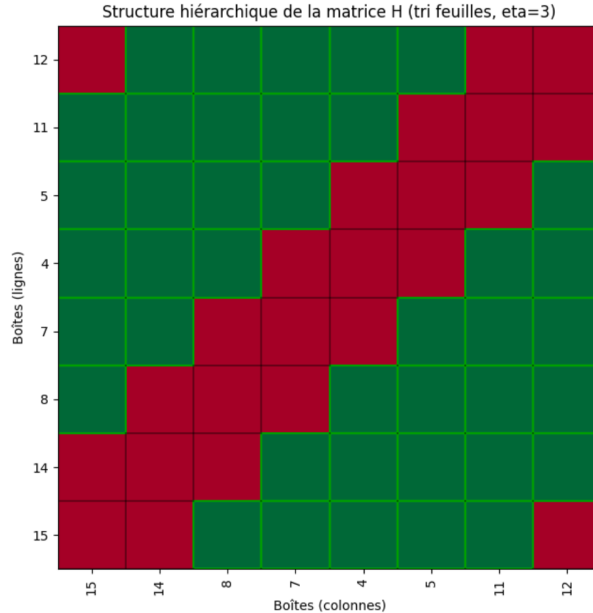


Figure 3 – Structure de la matrice hiérarchique, 100 points, niveau max = 3

Nous observons que les blocs non compressibles sont situés sur la diagonale, ainsi que dans les deux coins opposés. En effet, les blocs 12 et 15 sont adjacents sur le disque. Les blocs non compressibles correspondent donc bien aux blocs qui sont proches physiquement dans notre géométrie.

Afin de tester notre algorithme hiérarchique dans un cas plus complexe, nous avons également représenté la structure obtenue pour un maillage du disque à 400 points, en fixant le niveau max de l'arbre binaire à 5, comme présenté en Figure 4.

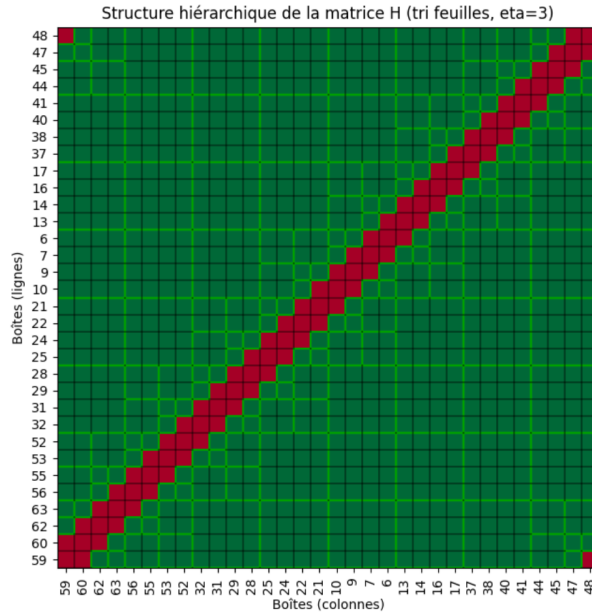


Figure 4 – Structure de la matrice hiérarchique, 400 points, niveau max = 5

Sur cette figure, on observe des blocs admissibles de différentes tailles, et en particulier une nécessité de raffiner la taille des blocs autour de la diagonale et des coins, car c'est là que se situent les singularités générées par la proximité des points.

Afin de vérifier que les blocs admissibles étaient bien de rang faible, nous avons calculé le rang numérique des sous-matrices associées dans la matrice A issue de notre code BEM, et l'avons comparé à la taille des sous-matrices.

Pour une matrice A de taille 400x400, $k = \pi$, avec niveau max = 10 et $N_{leaf} = 12$ (le critère limitant ici), nous avons obtenus 274 blocs admissibles et 144 blocs denses. Les blocs admissibles contenaient entre 7 et 11 points et étaient de rang 1 ou 2, avec un rang moyen sur tous les blocs admissibles de 1.5. En notant r le rang de chaque sous-matrice et $n \times n$ sa taille, on a bien $r < n/2$. Nous pouvons donc affirmer que tous les blocs admissibles sont de rang faible.

Pour évaluer le gain mémoire, nous avons calculé le taux de compression τ , défini comme le ratio entre le coût de stockage de la matrice sous forme hiérarchique et le coût de stockage de la matrice dense. Avec les paramètres ci-dessus, nous avons obtenu $\tau = \frac{54193}{160000} \approx 33.87\%$.

2.3 Analyse du Taux de Compression τ

Nous avons ensuite étudié l'évolution du taux de compression τ , défini comme le ratio entre le coût de stockage de la matrice sous forme hiérarchique et le coût de stockage de la matrice dense, en faisant varier la fréquence (via le nombre d'onde k) et la taille du maillage (N).

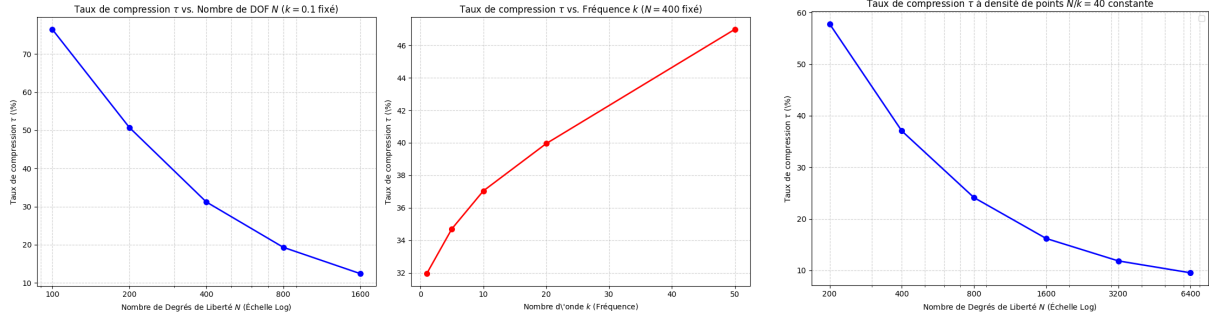


Figure 5 – Taux de compression τ en fonction de N et k . **Gauche** : Impact de k et de N séparément (complexité asymptotique). **Droite** : Impact de N à densité de points constante ($\frac{N}{k} = \text{constante}$).

- **Impact du Nombre de DOF (N)** : Lorsque k est fixé et que N augmente, le taux de compression τ diminue fortement. En effet, pour $k = 0.1$ et $N = 800$, $\tau \approx 19.33\%$, et pour $k = 0.1$ et $N = 1600$, $\tau \approx 12.43\%$. ce comportement correspond à ce qui était attendu, car quand le nombre de DOF augmente, la proportion de blocs compressibles augmente, et donc le coût de stockage de H-mat diminue.
- **Impact de la Fréquence (k)** : Lorsque N est fixé et que k augmente (par exemple, de $k = 10$ à $k = 50$ à $N = 400$), τ augmente. En effet, pour $k = 10$, $\tau \approx 37.04\%$, et pour $k = 50$, $\tau \approx 46.98\%$ en restant à $N = 400$. Ce comportement correspond à ce qui était attendu, car quand k augmente il y a plus d'oscillations et donc le rang des blocs admissibles augmente et donc le coût de stockage de H-mat augmente.
- **Densité de points constante** : Le taux de compression τ diminue de manière significative.

Ces résultats sont cohérent avec nos attentes, car ils démontrent l'efficacité sur des grands problèmes de la méthode des $\mathcal{H}$ -matrices et valident les hypothèses relatives à l'impact de la fréquence sur les rangs de compression. Nous n'expliquons en revanche pas pourquoi le taux de compression diminue sur une densité de points constant.

3 Application de la méthode H-mat à notre code BEM

Nous avons implémenté dans notre code BEM la méthode des H-matrices en utilisant l'ACA partiel pour obtenir une approximation de rang faible des blocs admissibles de A , couplée à un solveur itératif adapté (GMRES). L'ACA partiel permet de ne calculer que le bloc A_{XY} souhaité sans avoir besoin de calculer A en entier, ce qui permet d'économiser en termes de stockage et d'opérations effectuées.

Nous avons commencé par vérifier la cohérence de la trace obtenue avec H-mat et GMRES vis-à-vis de la trace que nous obtenions précédemment avec A dense, comme présenté en Figure 6.

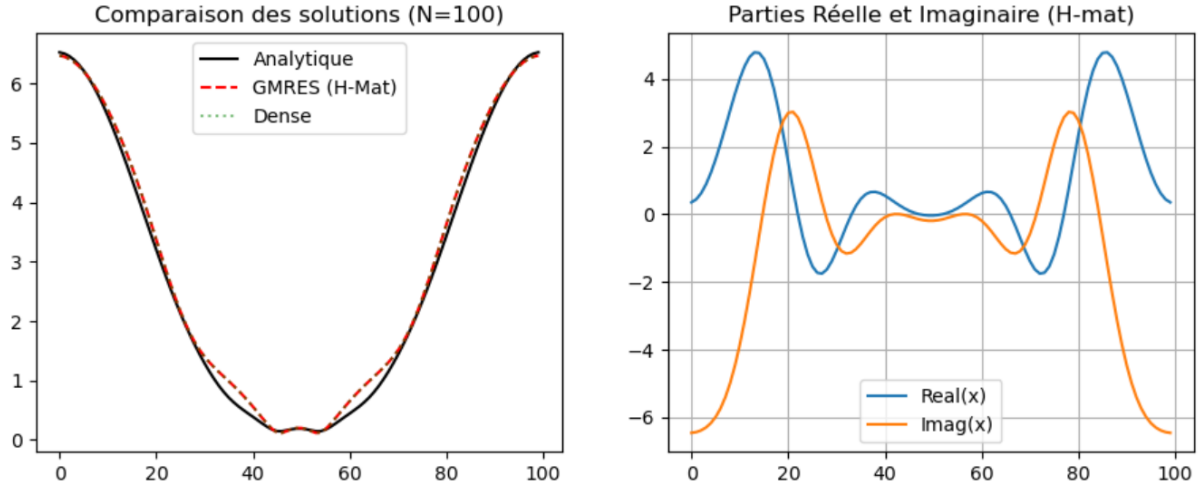


Figure 6 – À gauche : Comparaison de la trace obtenue par H-mat et GMRES avec la trace analytique et la trace obtenue par solveur direct avec A dense. À gauche : le module, à droite : la partie réelle et imaginaire de la trace obtenue par H-mat et GMRES.

Nous obtenons ainsi une très bonne approximation de la trace, pour un taux de compression de A valant $\tau \approx 84.48\%$ ($N = 100$, $k = \pi$). La figure de droite correspond précisément aux résultats obtenus dans nos précédents rapports.

4 Analyse de Performance et Complexité Algorithmique

Nous avons procédé à une analyse de performance en faisant varier le nombre d'inconnues N (le nombre de degrés de liberté) pour déterminer la taille maximale du problème que notre code est capable de traiter dans un temps raisonnable. Cette étude visait à comparer les temps de calculs entre la méthode rapide basée sur la **H-matrice** et le solveur itératif **GMRES**, avec la méthode classique utilisant l'assemblage **Dense** et un solveur direct.

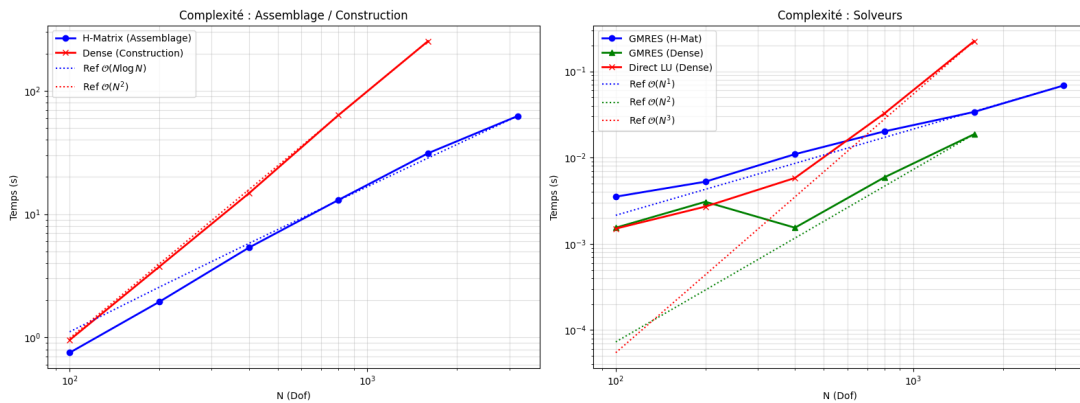


Figure 7 – Évolution des temps de calculs (log-log) en fonction du nombre d'inconnues N . Comparaison entre la méthode Dense et la méthode H-Matrice.

Le graphique (Figure 7), représentant le temps de calcul $\log(T)$ en fonction de la taille du problème $\log(N)$, valide les gains de performance apportés par la compression H-matrice.

On observe pour l'assemblage que :

- La construction de la **Matrice Dense** suit la complexité quadratique $\mathcal{O}(N^2)$.
- L'**Assemblage H-Matrice** suit la complexité quasi-linéaire $\mathcal{O}(N \log N)$.

Et pour la complexité que :

- La méthode par **Solveur Direct** sur matrice dense est la plus pénalisante avec une complexité cubique $\mathcal{O}(N^3)$.
- Le **GMRES** sur la matrice **Dense** est plus efficace ($\mathcal{O}(N^2)$) car il évite la factorisation complète, mais il reste dépendant de la taille non compressée de la matrice.
- L'utilisation de **GMRES** avec la **H-matrice** permet de tirer parti de la compression pour obtenir un produit matrice-vecteur rapide en $\mathcal{O}(N \log N)$. Ceci conduit à une complexité quasi-linéaire pour la résolution (pente légèrement meilleure de ≈ 1 dans cet exemple), validant l'approche H-matrice pour la gestion des problèmes de très grande taille.

4.1 Application du code BEM accéléré par la méthode H-mat à d'autres géométries

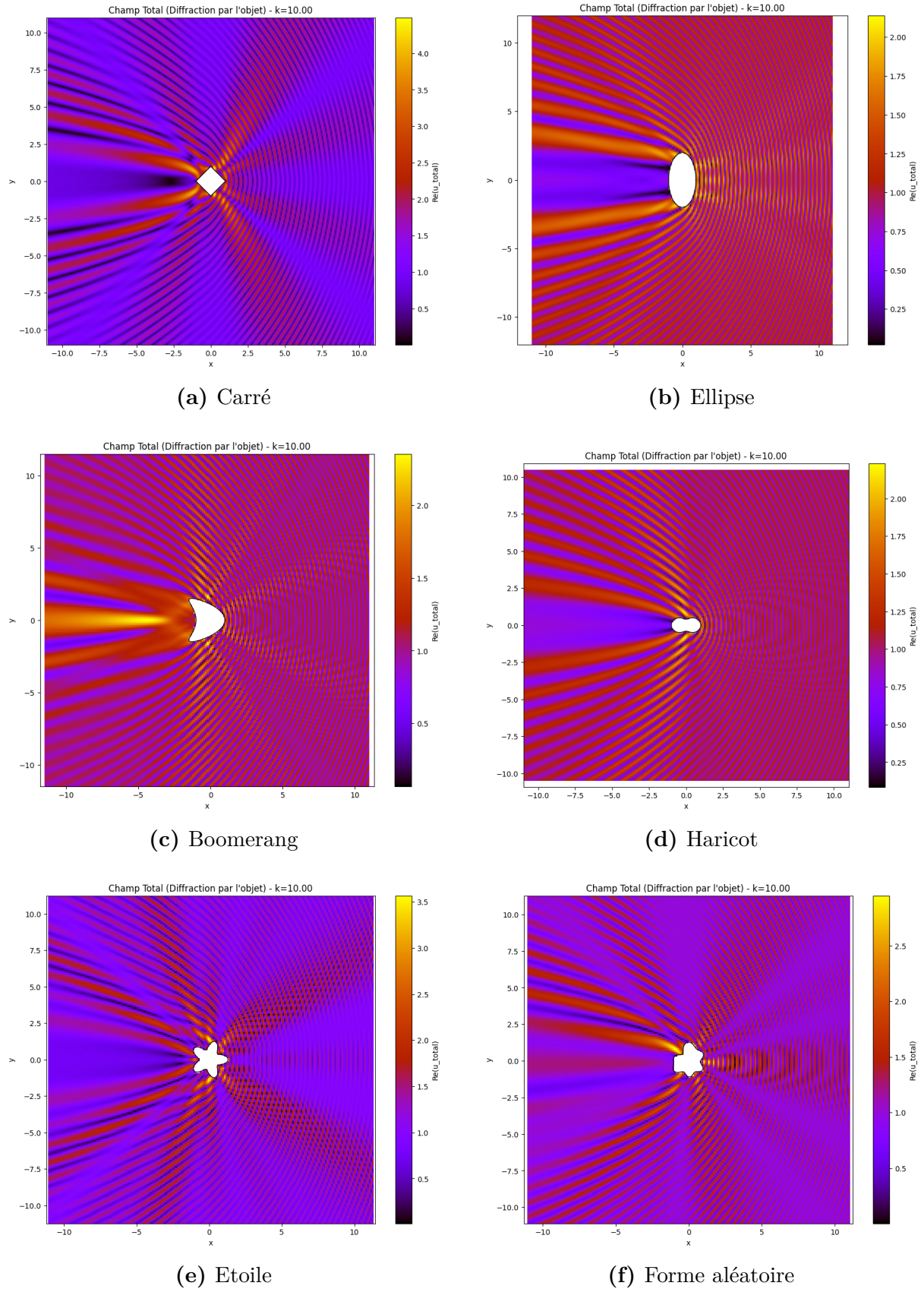


Figure 8 – Champ diffracté obtenu en considérant différents obstacles, pour $N = 800$ et $k = 10$.

Comme présenté en Figure 8, nous avons exploré comment notre code pouvait s'adapter à différentes géométries, comme le carré, l'ellipse, le boomerang, le haricot, l'étoile ou encore une géométrie générée aléatoirement en déformant un disque. Pour cela, il nous a fallu caractériser la frontière de chacune de ces géométries pour y définir un maillage. Une fois les coordonnées des noeuds obtenues, le reste du code fonctionnait comme dans le cas du disque. *Note intéressante* : Pour le carré, l'onde incidente "tape" le coin du carré, ce qui dégrade le conditionnement de la matrice (Singularité physique), ce qui entraîne une explosion du nombre d'itérations nécessaires au solveur GMRES. (On multiplie par plus de 100 le temps de calcul de GMRES)

5 Conclusion

La méthode des matrices hiérarchiques, couplée au solveur itératif GMRES, nous a permis d'optimiser notre code BEM afin de passer à une complexité en $N \log N$. Cela nous a permis d'améliorer considérablement les temps de calculs et le coût de stockage, tout en contrôlant la précision de l'approximation de la trace. Ainsi, nous avons pu traiter des problèmes de taille importante en un temps relativement court, ce qui représente un réel intérêt si l'on souhaite travailler sur des problèmes réalistes. Enfin, nous avons vu comment adapter notre code à d'autres géométries, démontrant qu'il pouvait s'appliquer à différentes situations de diffraction.

5.1 Annexe : Utilisation de l'IA

L'Intelligence Artificielle nous a permis d'accélérer la construction des codes pour certaines des méthodes, et à déboguer certains fragments de code. Elle nous a également servi à générer différentes géométries et à adapter le maillage de la frontière à celles-ci.